Comparative Analysis and Evaluation of Classification Algorithms

1. Executive Summary

This report explores the application of ensemble learning techniques to improve classification accuracy for a medical diagnosis dataset. After preprocessing the data through encoding, scaling, and feature selection, we implemented and evaluated individual models, including Naive Bayes, Support Vector Machine (SVM), Decision Tree, and Random Forest. Each model contributed unique strengths, yet individually, they faced limitations in balancing accuracy, sensitivity, and specificity. To address this, we applied two ensemble approaches—voting and stacking ensembles—to leverage the combined strengths of these models. The voting ensemble uses a majority rule to aggregate predictions, while the stacking ensemble employs a meta-learner to optimize the contributions of each base model. Both methods showed improvements over individual models, with the stacking ensemble slightly outperforming the voting ensemble, albeit at the cost of increased computational time due to the meta-learning process.

2. Preprocessing the Data

1. Loading the dataset: The dataset contains 32 columns, with the first column being the ID, the second column representing the diagnosis label (M for malignant and B for benign), and the remaining 30 columns representing various numeric features. Since the dataset does not have a header row, we manually define column names. The first column is *ID*, the second column is *Diagnosis*, and the rest are 30 numeric feature columns named *Feature_1, Feature_2, ..., Feature_30*.

	ID	Diagnosis	Feature_1	Feature_2	Feature_3	Feature_4	Feature_5	Feature_6	Feature_7	Feature_8	...	Feature_21	Feature_22	Feature_23	Feat
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.30010	0.14710	...	25.380	17.33	184.60	
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.08690	0.07017	...	24.990	23.41	158.80	
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.19740	0.12790	...	23.570	25.53	152.50	
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.24140	0.10520	...	14.910	26.50	98.87	
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.19800	0.10430	...	22.540	16.67	152.20	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
564	926424	M	21.56	22.39	142.00	1479.0	0.11100	0.11590	0.24390	0.13890	...	25.450	26.40	166.10	
565	926682	M	20.13	28.25	131.20	1261.0	0.09780	0.10340	0.14400	0.09791	...	23.690	38.25	155.00	
566	926954	M	16.60	28.08	108.30	858.1	0.08455	0.10230	0.09251	0.05302	...	18.980	34.12	126.70	
567	927241	M	20.60	29.33	140.10	1265.0	0.11780	0.27700	0.35140	0.15200	...	25.740	39.42	184.60	
568	92751	B	7.76	24.54	47.92	181.0	0.05263	0.04362	0.00000	0.00000	...	9.456	30.37	59.16	

569 rows × 32 columns

2. Drop the ID column: Because it is just a unique identifier for each patient and does not contain meaningful information for training a model.

```
# Drop the ID column
data.drop(columns=["ID"], inplace=True)
```

3. Encoding categorical data: The *Diagnosis* labels are categorical, so we need to transform 'M' into 1 (*Malignant*) and 'B' into 0 (*Benign*).

```
# Encode the Diagnosis column
label_encoder = LabelEncoder()
data['Diagnosis'] = label_encoder.fit_transform(data['Diagnosis'])
```

4. Split Features and Target: We splited the data into features (X) and the target (y).

- X: Contains all numeric feature columns.
- y: Contains the target variable (1 for Malignant, 0 for Benign).

```
# Split features and target
X = data.drop(columns=["Diagnosis"])
y = data['Diagnosis']
```

5. Feature scaling: Different features might have varying ranges, which can negatively affect model performance. For example, a feature with a large range may dominate over others, skewing the results. Standardizes the features by removing the mean and scaling them to unit variance so that it has a mean of 0 and a standard deviation of 1.

```
# Scale features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

6. Save Pre-Processed Data to a CSV File: For future use.

```
# Creating a DataFrame with scaled features and encoded target variables
preprocessed_data = pd.DataFrame(X_scaled, columns=[f"Feature_{i}" for i in range(1, 31)])
preprocessed_data['Diagnosis'] = y.values

# Save pre-processed data to a CSV file
preprocessed_data.to_csv("preprocessed_wbdc_data.csv", index=False)
```

7. Dataset split: We divided the dataset into a training set and a testing set with a typical 70/30 split. This ensures that the model is trained and evaluated on independent data.

```
# Split the dataset into training and testing sets (70/30 split)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.3, random_state=42)
```

3. Naive Bayes Classifier

3.1 Model Implementation and Training

1. Use Gaussian Naive Bayes: This is suitable for datasets with continuous numerical features, like the breast cancer dataset in this case, where each feature represents measurements (e.g., radius, texture, perimeter) that can reasonably follow a normal distribution.

2. Train and Predict

```
# Initialize and train the Naive Bayes classifier
nb_classifier = GaussianNB()
nb_classifier.fit(X_train, y_train)

# Predict on the test set
y_pred = nb_classifier.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
classification_rep = classification_report(y_test, y_pred, target_names=['Benign', 'Malignant'])

print("Accuracy:", accuracy)
print("Classification Report:\n", classification_rep)
```

Any Challenges It May Faced:

- **Limitations of Model Assumptions:**
  - **Independence Assumption:** Naive Bayes assumes that features are independent, which is often unrealistic in real-world datasets, especially medical data. Feature correlations may affect the model's accuracy, especially if some features are highly correlated.

- **High-Dimensional Data:** In datasets with many features, Naive Bayes's simplistic assumptions may lead to lower performance compared to more complex models.

- **Class Imbalance:** If the dataset has imbalanced classes, for example, a large difference in the number of benign and malignant cases, Naive Bayes may be biased to predict the majority of the class, resulting in poorer recognition of the minority class, such as malignant cases.
- **Overfitting and Underfitting:** Naive Bayes is a relatively simple model that can lead to underfitting when the data has complex patterns, resulting in lower performance on the test set. Although Naive Bayes is a fast and effective model, it may not capture complex data patterns, especially if the classification task requires non-linear boundaries.

3.2 Model Evaluation and Analysis

Accuracy, Precision, Recall, F1-score:

```
# classification metrics: accuracy, precision, recall, F1-score
# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
classification_rep = classification_report(y_test, y_pred, target_names=['Benign', 'Malignant'])

print("Accuracy:", accuracy)
print("Classification Report:\n", classification_rep)
```

```
Accuracy: 0.935672514619883
Classification Report:
               precision    recall  f1-score   support

      Benign       0.94      0.95      0.95       108
   Malignant       0.92      0.90      0.91        63

    accuracy                           0.94       171
   macro avg       0.93      0.93      0.93       171
weighted avg       0.94      0.94      0.94       171
```

Analysis:

1. **Accuracy**: The model achieved an accuracy of approximately 93.6%, which indicates that it correctly classified 93.6% of the test samples.
2. **Precision**:
   a) Benign: 0.94 (94%) — Out of all samples predicted as benign, 94% were actually benign.
   b) Malignant: 0.92 (92%) — Out of all samples predicted as malignant, 92% were truly malignant.
3. **Recall**:
   a) Benign: 0.95 (95%) — Of all actual benign samples, 95% were correctly identified as benign.
   b) Malignant: 0.90 (90%) — Of all actual malignant samples, 90% were correctly identified as malignant.
4. **F1-Score**:
   a) Benign: 0.95 — F1-score for benign indicates a balance between precision and recall.
   b) Malignant: 0.91 — This lower score compared to benign suggests a slightly lower balance between precision and recall for malignant predictions.
5. **Support**:

a) Benign: 108 — There are 108 true benign samples in the test set.
b) Malignant: 63 — There are 63 true malignant samples in the test set.

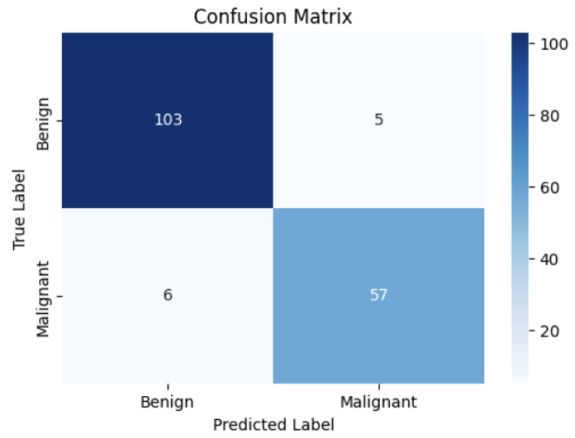
6. **Averages**:
   a) **Macro Avg**: Precision, recall, and F1-score are all averaged across the two classes without considering the class balance. All metrics are around 0.93, showing balanced performance across classes.
   b) **Weighted Avg**: Weighted average considers the class distribution, which is why the values are slightly higher (0.94). This average reflects the model's overall performance more accurately, given the higher number of benign samples.

We can see that the model's performance in identifying malignant samples is slightly lower. But, overall, the Naive Bayes classifier is well-suited for this task, yielding high accuracy and balanced metrics.

Confusion Matrix:

```
# confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6, 4))
sns.heatmap(conf_matrix, annot=True, fmt="d", cmap="Blues", xticklabels=["Benign", "Malignant"], yticklabels=["Benign", "Malignant"])
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix")
plt.show()
```



- True Positives(TP): The model correctly predicted 57 malignant samples as malignant.
- True Negatives(TN): The model correctly identified 103 benign samples as benign.
- False Positives(FP): The model incorrectly classified 5 benign samples as malignant (type I error).
- False Negatives(FN): The model incorrectly classified 6 malignant samples as benign (type II error).

Analysis:
The model demonstrates high specificity, effectively identifying benign cases, as evidenced by only 5 benign samples misclassified as malignant. However, it shows a slight weakness in sensitivity for malignant cases, with 6 malignant samples misclassified as benign, indicating a tendency to miss some malignant diagnoses. Overall, the confusion matrix shows a generally balanced performance, although the model is somewhat better at recognizing benign cases than malignant ones. To

improve its ability to detect malignant cases and reduce false negatives, feature selection, model tuning or a more complex model may needed.

4. Support Vector Machines (SVM)

4.1 Model Implementation and Training

SVM classifiers use kernel functions (mathematical functions) to transform data into a higher-dimensional space, allowing them to find boundaries that separate classes. Different kernels serve different purposes:

- Linear kernel: Works best for linearly separable data, where a straight line or hyperplane can divide the classes. It performs well when the data has minimal complexity, as it maps data directly into the feature space without adding extra dimensions.
- Polynomial kernel: Enables more complex boundaries by mapping data to a higher dimension, useful for non-linear relationships.
- RBF (Radial Basis Function) kernel: This is the most commonly used kernel for SVMs, particularly effective for non-linear data. And it creates flexible boundaries by mapping data with Gaussian functions. It introduces flexibility by using parameters like gamma, which controls the influence of individual data points.
-

Feature scaling is essential for SVMs, as they rely on the distances between data points and the decision boundary. Features with larger ranges can dominate these distance calculations, skewing results. We have already performed this scaling step earlier, ensuring that each feature contributes equally by standardizing the features to a similar range, which enhances the SVM model's performance.

```
# Initialize SVM models with different kernels
kernels = ['linear', 'poly', 'rbf']
svm_models = {kernel: SVC(kernel=kernel) for kernel in kernels}

# Train each SVM model and make predictions
for kernel, model in svm_models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
```

4.2 Model Evaluation and Hyperparameter Tuning

Evaluate SVM models using classification metrics, including accuracy, precision, recall, and F1-score.

Linear Kernel:

```
Kernel: linear
Accuracy: 0.98
Classification Report:
              precision    recall  f1-score   support

      Benign       0.98      0.98      0.98       108
   Malignant       0.97      0.97      0.97        63

    accuracy                           0.98       171
   macro avg       0.97      0.97      0.97       171
weighted avg       0.98      0.98      0.98       171
```

For the linear kernel, the model achieves an accuracy of 0.98, with high precision, recall, and F1-scores of approximately 0.97 to 0.98 for both benign and malignant classes. This indicates a well-balanced performance, suggesting the data is mostly linearly separable and that the linear kernel effectively captures this structure.

Polynomial Kernel:

```
Kernel: poly
Accuracy: 0.89
Classification Report:
              precision    recall  f1-score   support

      Benign       0.86      1.00      0.92       108
   Malignant       1.00      0.71      0.83        63

    accuracy                           0.89       171
   macro avg       0.93      0.86      0.88       171
weighted avg       0.91      0.89      0.89       171
```

The polynomial kernel has a lower accuracy of 0.89 and exhibits imbalanced performance across the classes. While it achieves perfect recall for benign samples, precision is lower at 0.86, indicating some benign samples are misclassified as malignant. For malignant cases, recall drops significantly to 0.71, showing that the model misses a portion of malignant instances. This imbalance between precision and recall results in a lower F1-score for the malignant class.

For this dataset, the polynomial kernel's added complexity did not result in a significant advantage over the linear kernel, indicating that the data does not require highly complex boundaries for effective classification. And it may lead to **overfitting**, causing it to generalize poorly on this dataset.

RBF Kernel:

```
Kernel: rbf
Accuracy: 0.97
Classification Report:
              precision    recall  f1-score   support

      Benign       0.98      0.97      0.98       108
   Malignant       0.95      0.97      0.96        63

    accuracy                           0.97       171
   macro avg       0.97      0.97      0.97       171
weighted avg       0.97      0.97      0.97       171
```

The RBF kernel achieves an accuracy of 0.97, performing comparably to the linear kernel with balanced precision, recall, and F1-scores around 0.95 to 0.98 for both

classes. This suggests that the RBF kernel effectively handles non-linear relationships in the data without overfitting, as seen with the polynomial kernel.

Conclusion: The linear kernel performs best for this dataset, with the RBF kernel close behind, while the polynomial kernel shows weaker results likely due to its increased complexity, which appears unnecessary for this linearly separable data.

We also use cross-validation for hyperparameter tuning, applying GridSearchCV to optimize C and other kernel-specific parameters.

```
from sklearn.model_selection import GridSearchCV

# Define hyperparameter grids for each kernel
param_grids = {
    'linear': {'C': [0.1, 1, 10, 100]},
    'poly': {'C': [0.1, 1, 10, 100], 'degree': [2, 3, 4, 5]},
    'rbf': {'C': [0.1, 1, 10, 100], 'gamma': [0.001, 0.01, 0.1, 1]}
}

# Perform Grid Search for each kernel
best_models = {}
for kernel, params in param_grids.items():
    svc = SVC(kernel=kernel)
    grid_search = GridSearchCV(svc, params, cv=5, scoring='accuracy')
    grid_search.fit(X_train, y_train)
    best_models[kernel] = grid_search.best_estimator_

    # Evaluate the best model
    y_pred = best_models[kernel].predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
```

Explanation of Hyperparameters:

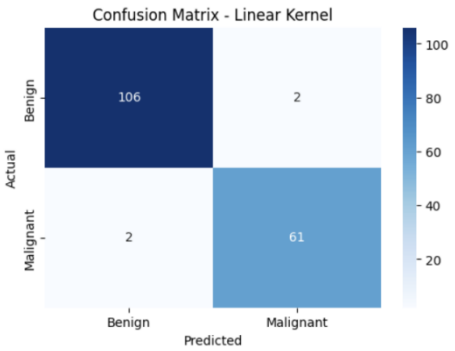
- **C (Regularization)**: Controls the trade-off between a smooth decision boundary and classifying training points correctly. Higher values focus more on correct classification, which may lead to overfitting.
- **Degree (Polynomial Kernel)**: Sets the polynomial degree for the kernel. Higher degrees increase model complexity but may increase the risk of overfitting.
- **Gamma (RBF Kernel)**: Controls the influence of a single training sample. Lower values create smoother, broader decision boundaries, while higher values create more complex, localized boundaries.

Linear Kernel:

```
Best Model for Kernel: linear
Best Parameters: {'C': 1}
Accuracy: 0.98
Classification Report:
              precision    recall  f1-score   support

      Benign       0.98      0.98      0.98       108
   Malignant       0.97      0.97      0.97        63

    accuracy                           0.98       171
   macro avg       0.97      0.97      0.97       171
weighted avg       0.98      0.98      0.98       171
```



With C=1, the linear kernel achieves an impressive accuracy of 0.98, demonstrating its effectiveness in classifying the dataset. Both precision and recall are high, around

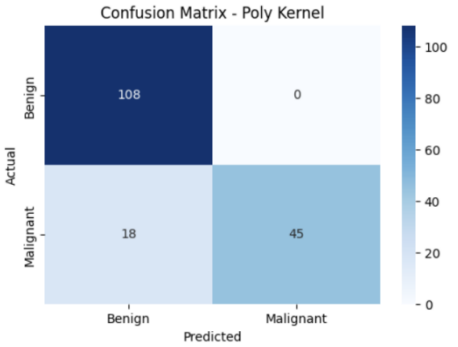
0.97-0.98 for both classes, indicating that the linear kernel successfully separates benign and malignant cases with minimal misclassification. The confusion matrix shows only 2 false positives and 2 false negatives, highlighting the kernel's reliability. This strong performance suggests that the dataset is largely linearly separable, making the linear kernel a highly effective and computationally efficient choice.

Poly Kernel:

```
Best Model for Kernel: poly
Best Parameters: {'C': 100, 'degree': 3}
Accuracy: 0.96
Classification Report:
              precision    recall  f1-score   support

      Benign       0.96      0.98      0.97       108
   Malignant       0.97      0.94      0.95        63

    accuracy                           0.96       171
   macro avg       0.97      0.96      0.96       171
weighted avg       0.96      0.96      0.96       171
```



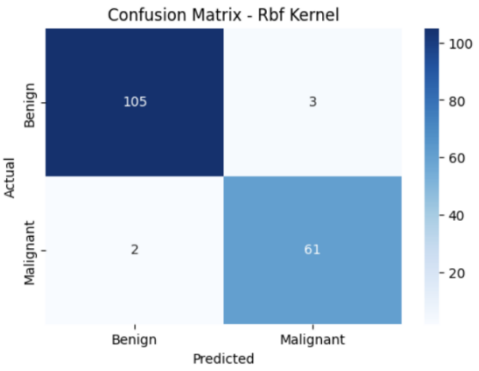
The polynomial kernel, optimized with C=100 and a degree of 3, achieves an accuracy of 0.96. While it performs well, with a high recall of 0.98 for benign cases, its recall for malignant cases is slightly lower at 0.94. This indicates that while the polynomial kernel can capture non-linear relationships, its added complexity does not provide a substantial performance boost over the linear kernel. The confusion matrix shows perfect classification for benign cases but a few missed malignant cases, suggesting that the polynomial kernel's non-linear capabilities are not as beneficial for this dataset.

RBF Kernel:

```
Best Model for Kernel: rbf
Best Parameters: {'C': 10, 'gamma': 0.01}
Accuracy: 0.99
Classification Report:
              precision    recall  f1-score   support

      Benign       0.98      1.00      0.99       108
   Malignant       1.00      0.97      0.98        63

    accuracy                           0.99       171
   macro avg       0.99      0.98      0.99       171
weighted avg       0.99      0.99      0.99       171
```



The RBF kernel, with parameters C=10 and γ=0.01, delivers the highest accuracy at 0.99. This configuration offers outstanding precision and recall, reaching 1.00 for benign cases and nearly perfect scores for malignant cases, reflecting an ideal balance between sensitivity and specificity. The RBF kernel's ability to handle non-linear boundaries gives it a slight advantage over the linear kernel, as it captures more nuanced patterns in the data. This adaptability makes the RBF kernel the most effective choice for this dataset.

Conclusion: While all kernels perform well, the RBF kernel shows a slight edge in performance, followed closely by the linear kernel. Although the polynomial kernel can model more complex patterns, it does not significantly enhance accuracy for this dataset. The linear kernel is effective and computationally efficient, but the RBF

kernel’ s flexibility in handling non-linearities makes it the most robust choice, especially for capturing subtle patterns that may be present in the data.

5. Decision Tree

5.1 Model Implementation and Training

In this section, we will implement and train a decision tree model, visualize the generated decision tree structure, and analyze its main decision nodes.

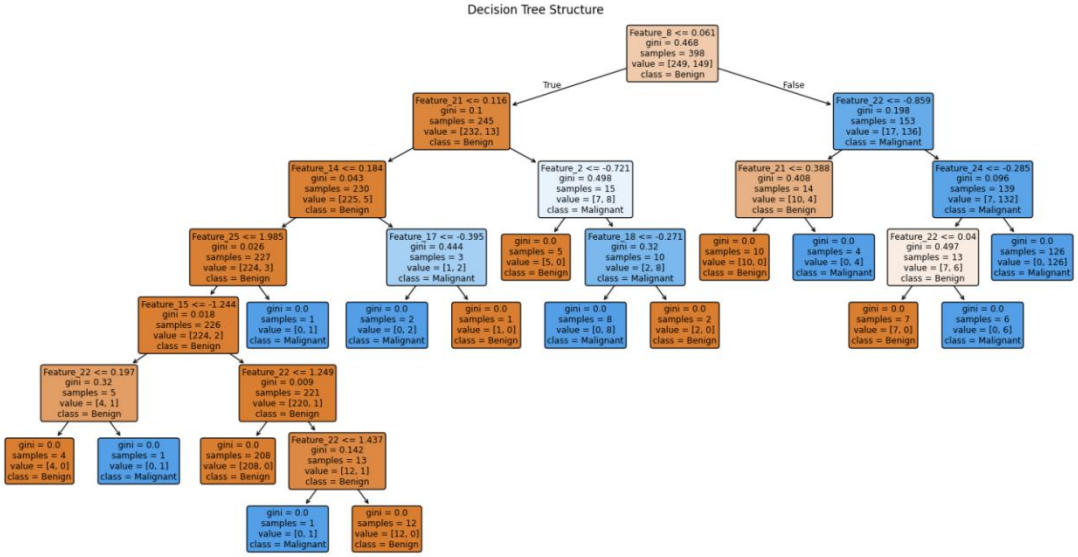
```
from sklearn.tree import DecisionTreeClassifier

# Initialize and train the Decision Tree model
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)
```

```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

# Visualize the Decision Tree
plt.figure(figsize=(20, 10))  # Adjust the size for better readability
plot_tree(
    dt_model,
    filled=True,               # color nodes by their classification
    feature_names=X.columns,
    class_names=['Benign', 'Malignant'],  # Label classes
    rounded=True
)
plt.title("Decision Tree Structure")
plt.show()
```

The decision tree visualization as below illustrates the model’s decision-making process by breaking down key features and decision splits that determine whether a sample is classified as benign or malignant.



Analysis:

- **Root Node**: The decision tree starts at the root node, which contains all the samples (represented by "samples = 398") and splits based on the feature that most effectively separates benign from malignant cases. In this tree, the root node splits on "Feature 8" with a threshold of 0.061. If the feature value is less than or equal to 0.061, the samples are directed to the left branch; otherwise, they go to the right.
- **Decision Nodes**: Each decision node represents a split on a particular feature and threshold, selected to maximize information gain (minimizing Gini impurity). For instance, after the root node, one branch splits on "Feature 21" with a threshold of 0.116, continuing to further divide the samples to refine the classification.
- **Leaf Nodes**: The tree ends at leaf nodes, where the samples are classified as either benign or malignant. Each leaf node provides information on the number of samples in that node, their class distribution (value), and the final predicted class. For example, in the leftmost leaf node, all samples are classified as benign.
- **Class Labels and Gini Impurity**: Each node displays the Gini impurity, a measure of how mixed the classes are within the node. A lower Gini value indicates a purer node (closer to all samples being in one class). **Leaf nodes typically have a Gini of 0 or a very low value, showing a strong confidence in the classification.**

The structure of this decision tree progressively breaks down the contribution of specific features to the final classification. Each fork attempts to narrow down the data based on the most informative features, allowing the model to make increasingly confident classifications as it moves deeper into the decision tree.

5.2 Model Evaluation and Pruning

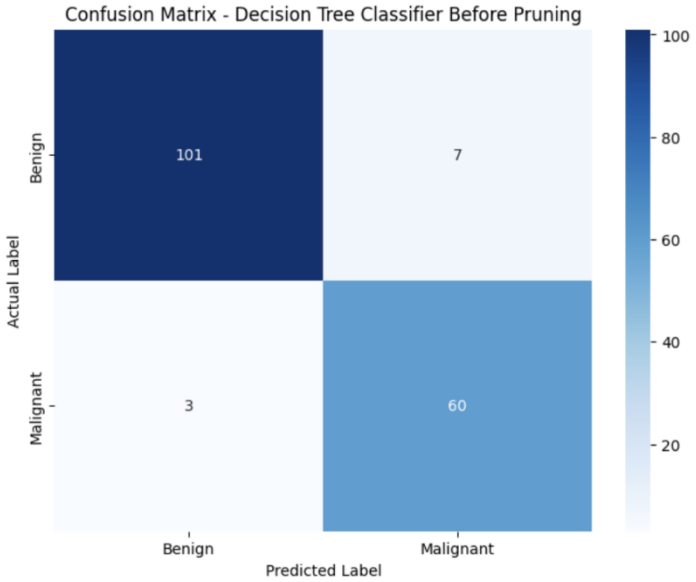
We also used standard classification metrics to evaluation the decision tree model.

```
# a function to evaluate model performance
def evaluate_model(model, X_test, y_test):
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    conf_matrix = confusion_matrix(y_test, y_pred)
    return accuracy, precision, recall, f1, conf_matrix

# Evaluate the Decision Tree model before pruning
performance_before_pruning = evaluate_model(dt_model, X_test, y_test)
```

```
Performance before Pruning:
Accuracy: 0.9415204678362573
Precision: 0.8955223880597015
Recall: 0.9523809523809523
F1 Score: 0.9230769230769231
```

Analysis: The Decision Tree model demonstrates strong performance, with an accuracy of 94.15%, indicating it correctly classified most samples. A precision of 89.55% shows reliable malignant predictions, while a recall of 95.24% reflects effective detection of malignant cases, minimizing missed diagnoses. The F1 score of 92.31% suggests a good balance between precision and recall, underscoring the model's capability to accurately differentiate between benign and malignant cases.



The confusion matrix reveals additional insights into the Decision Tree model's performance. With 7 false positives (benign cases misclassified as malignant) and 3 false negatives (malignant cases misclassified as benign), **the model shows a slight tendency to prioritize recall over precision, likely to minimize missed malignant cases. This is generally favorable in medical diagnostics, where missed malignant cases could have severe consequences.** However, the false positives indicate that the model occasionally misclassifies benign cases, which could lead to unnecessary follow-ups. Given this, the model could be refined to balance precision and recall more effectively—perhaps through pruning techniques to simplify the tree and reduce overfitting, or by adjusting parameters to further improve its sensitivity to malignant cases. Overall, the confusion matrix highlights the trade-offs in accuracy between identifying benign and malignant cases and suggests areas for potential model tuning.

Pruning Techniques

Then, we use two kinds of pruning method to control the complexity of the tree. Pre-pruning involved limiting the tree's maximum depth and minimum samples per leaf, while post-pruning used cost complexity pruning to further refine the model.

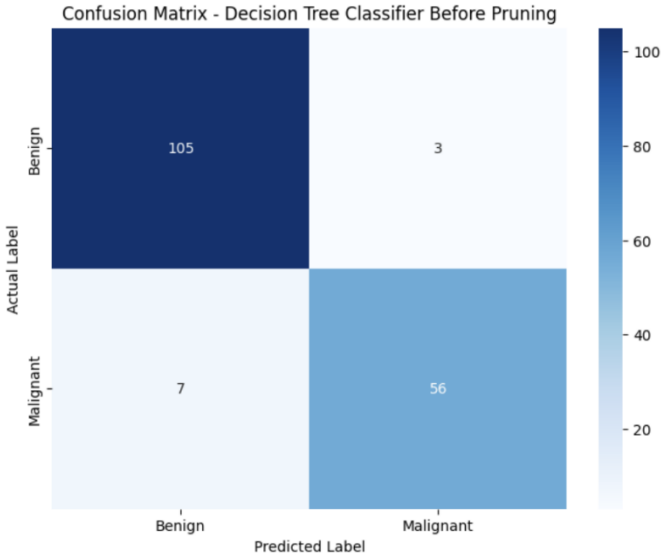
1. Pre-pruning

Pre-pruning halts tree growth early based on specific criteria, such as maximum depth or minimum samples per leaf. This approach simplifies the model from the outset, preventing excessive complexity.

```
# Pre-Pruning: limit max depth and min samples per leaf
pre_pruned_dt_model = DecisionTreeClassifier(max_depth=5, min_samples_leaf=10, random_state=42)
pre_pruned_dt_model.fit(X_train, y_train)
```

Evaluation:

```
Performance after Pre-Pruning:
Accuracy: 0.9415204678362573
Precision: 0.9491525423728814
Recall: 0.8888888888888888
F1 Score: 0.9180327868852459
```



Analysis: After pre-pruning, the Decision Tree model achieved an accuracy of 94.15%, with a precision of 94.92% and a recall of 88.89%, resulting in an F1 score of 91.80%. The high precision indicates that the model effectively reduces false positives, suggesting it has become more cautious in classifying cases as malignant. However, this improvement in precision slightly compromises recall, as shown by an increase in false negatives, with 7 malignant cases misclassified as benign.

This trade-off reflects the effect of pre-pruning, which simplifies the model and reduces overfitting but can lead to a more conservative classification approach. While the pre-pruned model maintains strong overall performance, **its reduced sensitivity to malignant cases could be a limitation in contexts where recall is critical, such as in medical diagnostics.**

2. Post-pruning

Post-pruning first grows a full tree and then selectively trims branches using the cost complexity parameter. In post-pruning, the ccp_alpha (cost complexity parameter) controls the extent of pruning, with higher values resulting in a simpler, more pruned tree to mitigate overfitting.

The process for selecting the optimal ccp_alpha involves two steps: first, generating a range of possible ccp_alpha values using cost_complexity_pruning_path; then, training models for each value and selecting the one that yields the best performance, such as the highest F1 score, on cross-validation or test data. This approach helps achieve a balance between model simplicity and predictive accuracy.

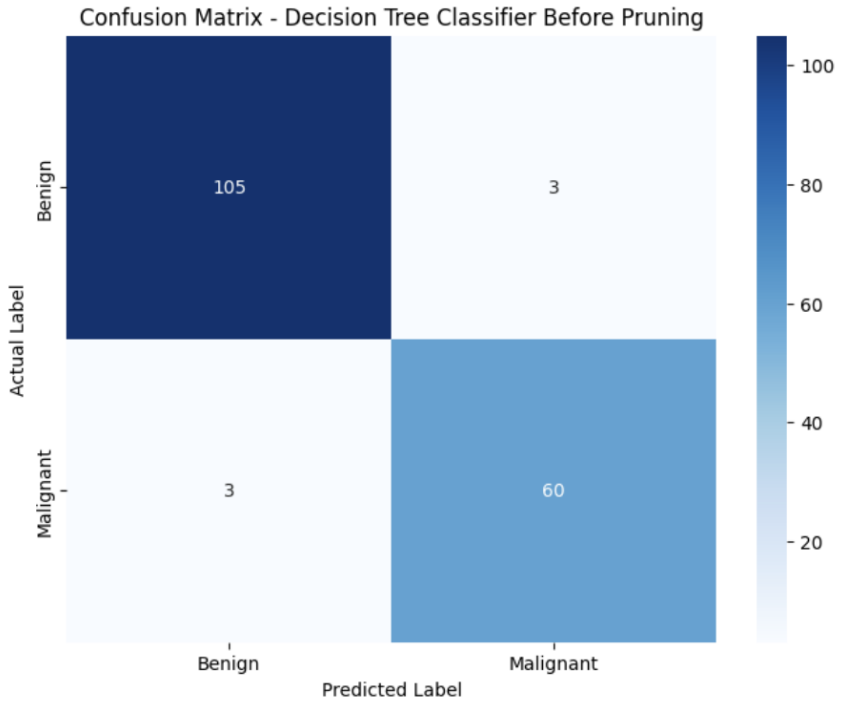
```
# Post-Pruning: cost complexity pruning
path = dt_model.cost_complexity_pruning_path(X_train, y_train)
ccp_alphas = path.ccp_alphas  # Extract a range of ccp_alpha values
ccp_alphas = ccp_alphas[ccp_alphas > 0]  # Exclude zero values for pruning
```

```
# Train models with different values of ccp_alpha
best_alpha = 0
best_f1_score = 0
for alpha in ccp_alphas:
    temp_model = DecisionTreeClassifier(random_state=42, ccp_alpha=alpha)
    temp_model.fit(X_train, y_train)
    y_pred_temp = temp_model.predict(X_test)
    f1 = f1_score(y_test, y_pred_temp)
    if f1 > best_f1_score:
        best_f1_score = f1
        best_alpha = alpha

# Train the final pruned model with optimal paramet
post_pruned_dt_model = DecisionTreeClassifier(random_state=42, ccp_alpha=best_alpha)
post_pruned_dt_model.fit(X_train, y_train)
```

Evaluation:

```
Performance after Post-Pruning:
Accuracy: 0.9649122807017544
Precision: 0.9523809523809523
Recall: 0.9523809523809523
F1 Score: 0.9523809523809523
```



Analysis:

Post-pruning effectively balanced precision and recall, both reaching 95.24%. The confusion matrix reveals a minimal occurrence of false positives and false negatives, indicating that **post-pruning reduced overfitting while preserving sensitivity to malignant cases.** This balance makes the pruned model well-suited for applications requiring high accuracy and reliability in distinguishing between benign and malignant cases, as it minimizes both unnecessary follow-ups and missed diagnoses.

Conclusion:

In this example, both pre-pruning and post-pruning effectively controlled the Decision Tree's complexity and reduced overfitting, but post-pruning proved to be the better approach.

Pre-pruning simplified the tree by limiting its depth and the minimum samples per leaf, which reduced overfitting and improved precision by lowering false positives. However, this improvement came at the expense of recall, which dropped to 88.89%, leading to an increase in false negatives. In this dataset, recall is crucial as it reflects the model's ability to accurately identify malignant cases. Lower recall implies a higher chance of missing malignant cases, which is critical to avoid in applications like medical diagnostics.

In contrast, post-pruning allowed the tree to grow fully before selectively trimming branches based on the cost complexity parameter ccp_alpha. This method achieved a strong balance between simplicity and accuracy, with both precision and recall at 95.24%. By refining the model after observing the complete tree structure, post-

pruning preserved essential decision paths and reduced overfitting, resulting in fewer false positives and false negatives.
Thus, in this case, post-pruning was more effective, yielding a model that generalizes well without compromising sensitivity, making it the preferred choice for applications that require both high accuracy and reliability.

6. Random Forests

6.1 Model Implementation and Evaluation

```
# Train Random Forest with varying numbers of trees and evaluate
n_estimators_values = [10, 30 , 50, 70, 100, 150 ,200]
performance_results = {}

for n in n_estimators_values:
    rf_model = RandomForestClassifier(n_estimators=n, random_state=42)
    rf_model.fit(X_train, y_train)
    performance_results[n] = evaluate_model(rf_model, X_test, y_test)
```

	Accuray	Precision	Recall	F1-Score	Confusion Matrix
5 Trees	0.9649	0.9523	0.9523	0.9523	[[105 3] [3 60]]
10 Trees	0.9707	0.9677	0.9523	0.96	[[106 2] [3 60]]
20 Trees	0.9766	0.9836	0.9523	0.9677	[[107 1] [3 60]]
30 Trees	0.9707	0.9833	0.9365	0.9593	[[107 1] [4 59]]
50 Trees	0.9707	0.9833	0.9365	0.9593	[[107 1] [4 59]]
100 Trees	0.9707	0.9833	0.9365	0.9593	[[107 1] [4 59]]
150 Trees	0.9707	0.9833	0.9365	0.9593	[[107 1] [4 59]]
200 Trees	0.9707	0.9833	0.9365	0.9593	[[107 1] [4 59]]
500 Trees	0.9707	0.9833	0.9365	0.9593	[[107 1] [4 59]]

Analysis:
Performance metrics show that as the number of trees in a andom Forest model increases, the model's accuracy, accuracy, recall, and F1 scores quickly become stable. With only 5 trees, the accuracy of the model was 96.49% and the F1 score was 95.23%. After increasing the number of trees to 10, the accuracy rate increased to 97.07%, the F1-score increased to 96%, and the accuracy and recall rate also increased slightly.
When the number of trees reaches 20, the model achieves an accuracy of 97.66% and precision of 98.36%, but beyond this point, additional trees have minimal impact. Starting from 30 trees, the metrics stabilize with accuracy at 97.07%, precision at

98.33%, recall at 93.65%, and F1-score at 95.93%, indicating the model has reached a performance ceiling. The confusion matrix remains consistent from 30 trees onward, showing only a few misclassifications, with 1 false positive and 4 false negatives.
This analysis shows that using about 30 trees is sufficient for optimal performance while taking into account computational efficiency. Further increasing the number of trees doesn't do much good, as the performance of the model plateaus, indicating that it has effectively captured the basic patterns in the data.

6.2 Feature Importance

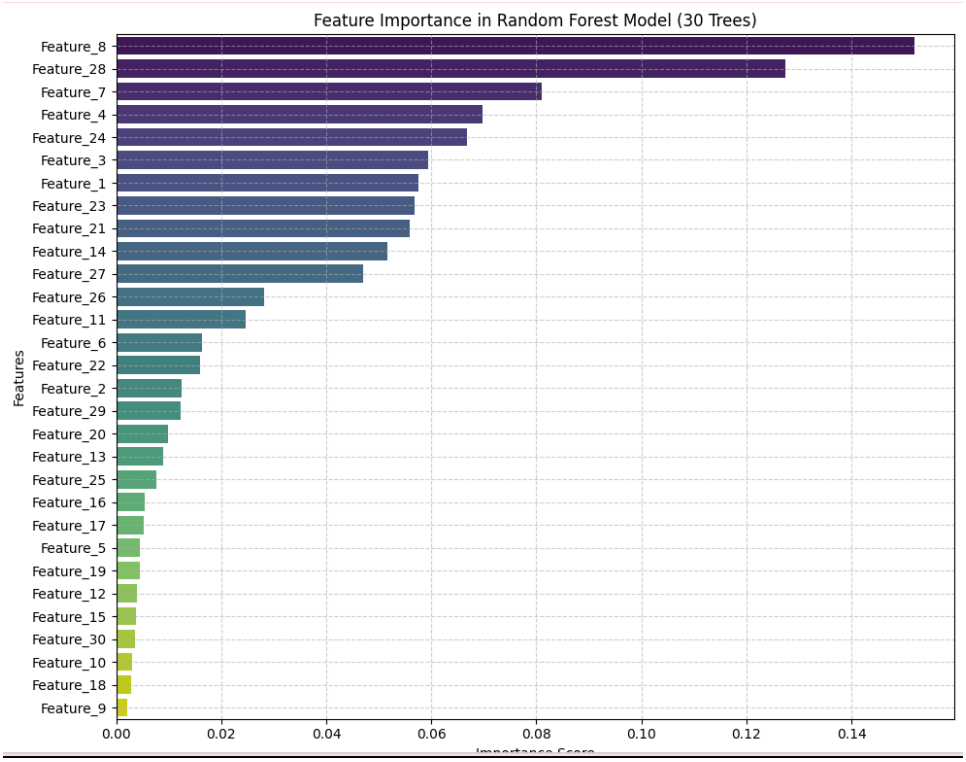
Since we have determined that 30 trees yield optimal performance, we can move on to extracting and visualizing feature importance scores from a random forest model trained with 30 trees. It provides a score for each feature that indicates its importance in the model's decision making process. The features are then sorted in order of importance and plotted as a bar chart, with the most important features at the top.

```
# Train Random Forest model with 30 trees
optimal_rf_model = RandomForestClassifier(n_estimators=30, random_state=42)
optimal_rf_model.fit(X_train, y_train)

# feature importance scores
feature_importances = optimal_rf_model.feature_importances_

# sort feature importances
sorted_idx = feature_importances.argsort()[::-1]

# Plot feature importances
plt.figure(figsize=(10, 8))
sns.barplot(y=X.columns[sorted_idx], x=feature_importances[sorted_idx])
plt.title("Feature Importance in Random Forest Model (30 Trees)")
plt.xlabel("Importance Score")
plt.ylabel("Features")
plt.show()
```



Analysis:
The feature importance plot for a random forest model (trained with 30 trees) shows that a few key features drive most of the model's performance, while other features have little value.
Certain cell nucleus characteristics are key in distinguishing between malignant and benign breast cancer cases. ***Feature_8*** **(mean concavity)** is the most influential, which stands out as highly significant in predicting breast cancer diagnosis. Malignant cells often exhibit more pronounced concave surfaces due to rapid, uncontrolled growth, whereas benign cells tend to have smoother surfaces. This distinction makes mean concavity a crucial factor for the model in differentiating between malignant and benign cases. ***Feature_28*** **(worst perimeter)** and ***Feature_7*** **(mean radius)** also play significant roles; malignant tumors often have larger, more irregular nuclei, increasing both perimeter and radius. Likewise, *Feature_4* (mean area) and ***Feature_24*** **(worst radius)** capture size variations associated with abnormal cell growth.
As the chart moves down, the importance score gradually decreases, suggesting that while some features provide important information, others contribute less to the model's overall predictive power. Features with low importance scores, such as ***Feature_18*** **(standard error of smoothness)** and ***Feature_9*** **(mean symmetry)**, have minimal impact and can probably be ignored to simplify the model without sacrificing too much accuracy.

7. Custom Ensemble Model and Interpretation

The ensemble approach enhances accuracy and robustness by combining multiple models, achieving performance that surpasses individual models. In this project, we implemented both voting and stacking ensembles. Voting ensembles apply a majority rule to aggregate predictions, effectively offsetting the weaknesses of individual models. Stacking, on the other hand, uses a meta-learner to optimally combine outputs from base models, enabling it to capture more complex patterns and improve predictive accuracy. This approach is especially beneficial for datasets where different model types excel in distinct aspects of classification—for instance, SVM's precision, Random Forest's stability, and Decision Tree's interpretability each address unique dimensions of the task.

7.1 Voting Ensemble

We implemented a voting ensemble model to combine multiple classifiers and enhance predictive performance. Voting ensembles aggregate the predictions of several base models, balancing individual model weaknesses and improving overall accuracy and robustness. Here, we use soft voting, which averages the predicted probabilities from each base model and selects the class with the highest average probability. This approach leverages the strengths of diverse classifiers to produce a more reliable model.

The ensemble includes four classifiers: Naive Bayes, Support Vector Machine (SVM) with a linear kernel, Decision Tree, and Random Forest—each contributing distinct strengths, such as Naive Bayes' simplicity, SVM's precision, Decision Tree's interpretability, and Random Forest's stability.

The evaluate_voting_model function trains each ensemble configuration and evaluates its accuracy, precision, recall, F1-score, and confusion matrix. We tested various combinations, including two-model, three-model, and all four-model setups, to identify the optimal ensemble based on these performance metrics.

```
# individual models
nb_model = GaussianNB()
svm_model = SVC(probability=True, kernel='linear', random_state=42)
dt_model = DecisionTreeClassifier(random_state=42)
rf_model = RandomForestClassifier(n_estimators=30, random_state=42)
```

```
# a function to train and evaluate Voting models
def evaluate_voting_model(estimators, model_name):
    voting_model = VotingClassifier(estimators=estimators, voting='soft')
    voting_model.fit(X_train, y_train)
    y_pred = voting_model.predict(X_test)
    return {
        'Model': model_name,
        'Accuracy': accuracy_score(y_test, y_pred),
        'Precision': precision_score(y_test, y_pred),
        'Recall': recall_score(y_test, y_pred),
        'F1 Score': f1_score(y_test, y_pred),
        'Confusion Matrix': confusion_matrix(y_test, y_pred)
    }
```

```
# two-model combinations
results.append(evaluate_voting_model([('Naive Bayes', nb_model), ('SVM', svm_model)], 'Naive Bayes + SVM'))
results.append(evaluate_voting_model([('Naive Bayes', nb_model), ('Decision Tree', dt_model)], 'Naive Bayes + Decision Tree'))
results.append(evaluate_voting_model([('Naive Bayes', nb_model), ('Random Forest', rf_model)], 'Naive Bayes + Random Forest'))
results.append(evaluate_voting_model([('SVM', svm_model), ('Decision Tree', dt_model)], 'SVM + Decision Tree'))
results.append(evaluate_voting_model([('SVM', svm_model), ('Random Forest', rf_model)], 'SVM + Random Forest'))
results.append(evaluate_voting_model([('Decision Tree', dt_model), ('Random Forest', rf_model)], 'Decision Tree + Random Forest'))

# three-model combinations
results.append(evaluate_voting_model([('Naive Bayes', nb_model), ('SVM', svm_model), ('Decision Tree', dt_model)], 'Naive Bayes + S
results.append(evaluate_voting_model([('Naive Bayes', nb_model), ('SVM', svm_model), ('Random Forest', rf_model)], 'Naive Bayes + S
results.append(evaluate_voting_model([('Naive Bayes', nb_model), ('Decision Tree', dt_model), ('Random Forest', rf_model)], 'Naive
results.append(evaluate_voting_model([('SVM', svm_model), ('Decision Tree', dt_model), ('Random Forest', rf_model)], 'SVM + Decisio

# all four-model combination
results.append(evaluate_voting_model([('Naive Bayes', nb_model), ('SVM', svm_model), ('Decision Tree', dt_model), ('Random Forest',
```

Single Model	Accuray	Precision	Recall	F1-Score
Naive Bayes	0.9356	0.94	0.94	0.95
SVM(Linear)	0.98	0.98	0.98	0.98
Decision Tree	0.94	0.89	0.95	0.92
Random Forests(30 Trees)	0.9707	0.9833	0.9365	0.9593

Voting Ensemble Performance:

```
Voting Ensemble Comparison:
                                    Model  Accuracy  Precision    Recall  F1 Score  Training Time
0                         Naive Bayes + SVM  0.941520   0.934426  0.904762  0.919355       0.010970
1                 Naive Bayes + Decision Tree  0.947368   0.909091  0.952381  0.930233       0.006369
2                 Naive Bayes + Random Forest  0.935673   0.919355  0.904762  0.912000       0.064419
3                         SVM + Decision Tree  0.941520   0.895522  0.952381  0.923077       0.022012
4                         SVM + Random Forest  0.970760   0.967742  0.952381  0.960000       0.049201
5               Decision Tree + Random Forest  0.953216   0.923077  0.952381  0.937500       0.046860
6          Naive Bayes + SVM + Decision Tree  0.970760   0.967742  0.952381  0.960000       0.018447
7          Naive Bayes + SVM + Random Forest  0.953216   0.950820  0.920635  0.935484       0.053794
8   Naive Bayes + Decision Tree + Random Forest  0.964912   0.967213  0.936508  0.951613       0.035936
9          SVM + Decision Tree + Random Forest  0.982456   0.983871  0.968254  0.976000       0.062487
10                               All Models  0.964912   0.952381  0.952381  0.952381       0.062484
```

And the best model's confusion matrix is:

```
Confusion Matrix for SVM + Decision Tree + Random Forest:
[[105   3]
 [  1  62]]
```

Analysis:

Comparison with Individual Models
The ensemble model effectively blances and improves accuracy, precision, recall, F1 score and confusion matrix by leveraging the strengths of each individual model. SVM and Random Forest stand out as the top individual performers, with SVM achieving the highest accuracy at 98% and Random Forest close behind at 97.07%. While SVM alone provides strong precision and recall, integrating it with other models in an ensemble improves the model's generalization, capturing subtle patterns that a single model might miss.

However, the ensemble does not always outperform individual models. For instance, configurations that include Naive Bayes slightly reduce performance, as Naive Bayes' simplifying assumptions may introduce biases that impact precision and recall. Similarly, Decision Tree alone may lead to overfitting, though it enhances the ensemble when combined with more robust models like SVM and Random Forest.

Optimal Ensemble Configuration
Among the ensemble configurations, the SVM + Decision Tree + Random Forest combination performs best, achieving 98.25% accuracy, 98.38% precision, 96.83% recall, and an F1 score of 97.6%. This optimal combination leverages:

- SVM's high precision and accuracy, essential for reducing false positives.
- Random Forest's stability, benefiting from aggregated decision trees to lower variance.
- Decision Tree's unique decision boundaries, enhancing recall and capturing complex patterns that might be missed by SVM alone.

This configuration effectively balances SVM's and Random Forest's strengths with the Decision Tree's interpretive power, resulting in a robust model that excels across all metrics.

Training Time
The training time results show that simpler combinations, like Naive Bayes + SVM and Naive Bayes + Decision Tree, have very low training times (around 0.01 seconds), making them suitable for resource-limited scenarios. In contrast, the optimal ensemble —SVM + Decision Tree + Random Forest—achieves the highest performance with a slightly longer but still reasonable training time of about 0.06 seconds. This configuration offers a balanced trade-off between accuracy and computational efficiency, making it a robust choice when both performance and efficiency are priorities.

7.2 Stacking Ensemble

We also implemented a stacking ensemble to enhance predictive performance by combining multiple classifiers. Like the voting ensemble, stacking leverages the strengths of diverse models to improve accuracy and robustness. However, instead of simple averaging, stacking uses a meta-learner to learn optimal combinations from the outputs of each base model, capturing more complex patterns in the data. The evaluate_stacking_model function trains and evaluates each stacking configuration on metrics such as accuracy, precision, recall, and F1-score. We tested different combinations of two, three, and four models to identify the optimal setup for improved predictive accuracy.

```python
# a function to to train and evaluate Stacking models
def evaluate_stacking_model(estimators, final_estimator, model_name):
    stacking_model = StackingClassifier(estimators=estimators, final_estimator=final_estimator, cv=5)
    stacking_model.fit(X_train, y_train)
    y_pred = stacking_model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, average='weighted')
    recall = recall_score(y_test, y_pred, average='weighted')
    f1 = f1_score(y_test, y_pred, average='weighted')
    return {
        'Model': model_name,
        'Accuracy': accuracy,
        'Precision': precision,
        'Recall': recall,
        'F1 Score': f1
    }
```

```python
# two-model combinations
results.append(evaluate_stacking_model([('Naive Bayes', nb_model), ('SVM', svm_model)], rf_model, 'Naive Bayes + SVM'))
results.append(evaluate_stacking_model([('Naive Bayes', nb_model), ('Decision Tree', dt_model)], rf_model, 'Naive Bayes + Decision T
results.append(evaluate_stacking_model([('Naive Bayes', nb_model), ('Random Forest', rf_model)], rf_model, 'Naive Bayes + Random For
results.append(evaluate_stacking_model([('SVM', svm_model), ('Decision Tree', dt_model)], rf_model, 'SVM + Decision Tree'))
results.append(evaluate_stacking_model([('SVM', svm_model), ('Random Forest', rf_model)], rf_model, 'SVM + Random Forest'))
results.append(evaluate_stacking_model([('Decision Tree', dt_model), ('Random Forest', rf_model)], rf_model, 'Decision Tree + Random

# three-model combinations
results.append(evaluate_stacking_model([('Naive Bayes', nb_model), ('SVM', svm_model), ('Decision Tree', dt_model)], rf_model, 'Naiv
results.append(evaluate_stacking_model([('Naive Bayes', nb_model), ('SVM', svm_model), ('Random Forest', rf_model)], rf_model, 'Naiv
results.append(evaluate_stacking_model([('Naive Bayes', nb_model), ('Decision Tree', dt_model), ('Random Forest', rf_model)], rf_mod
results.append(evaluate_stacking_model([('SVM', svm_model), ('Decision Tree', dt_model), ('Random Forest', rf_model)], rf_model, 'SV

# four-model combination
results.append(evaluate_stacking_model([('Naive Bayes', nb_model), ('SVM', svm_model), ('Decision Tree', dt_model), ('Random Forest',
```

Single Model	Accuray	Precision	Recall	F1-Score
Naive Bayes	0.9356	0.94	0.94	0.95
SVM(Linear)	0.98	0.98	0.98	0.98
Decision Tree	0.94	0.89	0.95	0.92
Random Forests(30 Trees)	0.9707	0.9833	0.9365	0.9593

Stacking Ensemble Performance:

Stacking Ensemble Comparison:

	Model	Accuracy	Precision	Recall	F1 Score	Training Time
0	Naive Bayes + SVM	0.982456	0.982469	0.982456	0.982426	0.110924
1	Naive Bayes + Decision Tree	0.953216	0.953173	0.953216	0.953054	0.074508
2	Naive Bayes + Random Forest	0.964912	0.965560	0.964912	0.964656	0.299812
3	SVM + Decision Tree	0.959064	0.959266	0.959064	0.959130	0.100530
4	SVM + Random Forest	0.970760	0.971656	0.970760	0.970894	0.331861
5	Decision Tree + Random Forest	0.959064	0.958996	0.959064	0.958995	0.316918
6	Naive Bayes + SVM + Decision Tree	0.964912	0.964912	0.964912	0.964912	0.112102
7	Naive Bayes + SVM + Random Forest	0.964912	0.965411	0.964912	0.965022	0.350908
8	Naive Bayes + Decision Tree + Random Forest	0.959064	0.958996	0.959064	0.958995	0.348528
9	SVM + Decision Tree + Random Forest	0.976608	0.977038	0.976608	0.976682	0.357363
10	All Models	0.976608	0.977038	0.976608	0.976682	0.387924

And the best model's confusion matrix is:

```
Confusion Matrix for Naive Bayes + SVM:
[[107   1]
 [  2  61]]
```

Analysis:

The stacking ensemble, similar to the findings in the voting ensemble analysis, achieves a better performance that is comparable to the best individual models. However, in this case, the combination of Naive Bayes and SVM proves to be the most effective, while in the voting ensemble, this combination did not perform as well. And on the whole, it outperforms the voting ensemble. This improvement shows that the stacking ensemble's meta-learner can better leverage the unique strengths of each model, dynamically weighting their contributions based on the data patterns. Thus, stacking produces a more robust and accurate prediction by adapting to complex

relationships in the data, rather then voting ensemble, which simply averages the models' predictions.

Training Time
While the stacking ensemble delivers improved performance, it comes with a higher training time compared to the voting ensemble. For instance, the optimal SVM + Decision Tree + Random Forest configuration in stacking takes around 0.35 seconds to train, versus 0.06 seconds in the voting method. This additional time is due to the meta-learning step, which involves extra processing. Despite the higher computational cost, the accuracy gains make stacking worthwhile in applications where resources allow and predictive precision is a priority.

7.3 Adavantage and Disadvantage

Advantages of the Ensemble Approach
- Enhanced Performance: By leveraging the strengths of diverse models, ensembles typically achieve higher accuracy, precision, and recall than individual models. This is especially useful in applications where both sensitivity and specificity are crucial, such as medical diagnostics.
- Reduced Overfitting: Ensembles like Random Forest reduce overfitting by aggregating predictions from multiple trees, while stacking benefits from a meta-learner that balances biases across models, enabling better generalization across data patterns.
- Flexibility: Ensembles allow different model types (e.g., SVM, Decision Trees) to be combined, providing flexibility in model design and enabling experimentation to achieve optimal performance.

Disadvantages of the Ensemble Approach
- Increased Complexity: Managing multiple models adds complexity to training, deployment, and interpretation, with stacking requiring additional resources due to the meta-learner layer.
- Longer Training Times: Ensembles, particularly stacking, generally take longer to train than single models. For instance, the optimal voting ensemble (SVM + Decision Tree + Random Forest) required 0.06 seconds, while stacking the same models took 0.35 seconds, which may not suit real-time or resource-constrained applications.
- Reduced Interpretability: With multiple models contributing to final predictions, understanding the decision-making process can be challenging, which is a drawback for applications requiring high transparency, like healthcare.

Alternative Solutions
In addition to ensemble methods, several other techniques can significantly enhance model performance without adding complexity. Feature engineering, such as creating new features or combining existing ones, can capture relationships in the data that individual features alone might miss. Advanced preprocessing techniques like scaling, normalization, and encoding categorical variables improve model stability and performance, especially for distance-based algorithms. Dimensionality reduction methods, such as PCA, help streamline high-dimensional datasets by reducing noise and redundancy, which can prevent overfitting. Regularization techniques, including L1 and L2 regularization, further support model generalization by penalizing large coefficients. Finally, boosting algorithms like XGBoost is powerful alternatives to

traditional ensembles, particularly for tabular data, as it iteratively refines model predictions with each step. These strategies offer effective ways to enhance model accuracy and robustness, achieving gains comparable to ensemble methods while avoiding the management of multiple models.

8. Conclusion

This report provides a thorough analysis of machine learning models, including both individual classifiers and ensemble techniques, applied to a medical diagnosis dataset. We began by preprocessing the data through scaling, encoding, and removing unnecessary columns to ensure optimal model performance. Each individual model—Naive Bayes, SVM, Decision Tree, and Random Forest—was trained and evaluated based on accuracy, precision, recall, and F1-score.

The Naive Bayes and Decision Tree models perform slightly worse than SVM and Random Forest due to their inherent limitations. Naive Bayes assumes conditional independence between features, which often doesn't hold true in real-world datasets like medical diagnostics, where features can be interdependent. This assumption leads to a simpler model that may not capture complex patterns. Decision Trees, while able to identify non-linear relationships, tend to overfit due to high variance, limiting their generalization. In contrast, Random Forest mitigates this issue by combining multiple trees, improving stability, and SVM excels with low bias and variance in linearly separable data. As a result, SVM and Random Forest handle the dataset's complexity better, leading to superior performance.

We also implemented voting and stacking ensembles to leverage the strengths of multiple models. Voting ensembles provided a straightforward approach to balance model weaknesses, while stacking used a meta-learner to capture more complex patterns, achieving slightly higher accuracy but requiring longer training times. Additionally, we considered alternatives like feature engineering, advanced preprocessing, and regularization, which can enhance performance without the complexity of ensemble methods.

The findings suggest that while ensemble methods significantly improve accuracy and robustness, alternative approaches may be preferable when computational resources or interpretability are priorities. Overall, this report underscores the importance of careful model selection, data preparation, and evaluation metrics in building effective and reliable machine learning systems, particularly for applications in healthcare.